ARM

一、Basic

(一) Concept

1、从 transaction/relation DB & other information repositories 存储库 中, finding frequent 频率 pattern, associations 关联, correlations 相关, or casual 因果 structure

2、Frequent Pattern：在DB中 frequent 出现的 pattern

ex：a set of items (什么东西一起买)

subsequence (买完A，再买B)

substructure (该drug与DNA有何关联)

↳ why? diclose an intrinsic 内在 and important property of data set which is fundamental of association/sequency/classification/clustering/data warehouse

(二) Measure

1、Rule Form：Body1 ^ Body2 ⇒ Head [support, confidence]

2、wildcard：* ⇒ B hint：通过什么可以提高B的销量

A ⇒ * hint：A的销量增加，可以再顺带卖些什么

3、$Supp(X \Rightarrow Z) = P(X \cap Z)$ —— statistical significance

$Conf(X \Rightarrow Z) = P(Z|X)$ —— rule's strength

Transaction ID	Items Bought
1000	A,B,C
2000	A,C
3000	A,D
4000	B,E,F

Let minimum support 50%, and minimum confidence 50%, we have

- A ⇒ C (50%, 66.6%)
- C ⇒ A (50%, 100%)

{ 保留 ≥ min_support
{ 丢弃 < min_support

筛选满足最小支撑度 + 最小置信度

4、Variants of Association Rule Mining 关联规则挖掘变体

① Boolean vs Quantitative (属性是数值，区间划分，ex：age："30~39")

② single dimen vs multiple dimen (多维 ex：age, salary 等)

③ single level vs multiple level analysis (什么牌子的beer与什么牌子的尿布)

④ i correlation, causality analysis

ii inter-transaction association rule mining (不是同一次交易，对不同交易之间)

iii sequential association rule mining (时间顺序)

iv constraints enforced (带约束挖掘，ex：小订单 (sum<100) 是否会触发大额采购 (sum>100)

二、AR Mining Algorithm —— Apriori

1、Any subset of a frequent itemset (ex：{A,B}中的{A} and {B}) must be frequent.

逆否：若一个项集存在非频繁子集，那该项集本身也是非频繁的，可以直接删掉

2、two key steps

① Mining frequent items：迭代寻找 1项集，2项集，…，n项集

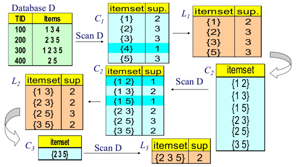
② Rule Generation：利用①得到的 frequent item，计算 $conf(X \Rightarrow Z)$

3、实例

$C_k \xrightarrow{prune} L_k \xrightarrow{join} C_{k+1}$ k表示集的维度

C_k：—— 不满足 min_support 每轮人为设定

L_k：—— 满足 min_support



Database D

TID	Items
100	1 3 4
200	2 3 5
300	1 2 3 5
400	2 5

C_1

itemset	sup.
{1}	2
{2}	3
{3}	3
{4}	1
{5}	3

L_1

itemset	sup.
{1}	2
{2}	3
{3}	3
{5}	3

C_2

itemset
{1 2}
{1 3}
{1 5}
{2 3}
{2 5}
{3 5}

C_2

itemset	sup
{1 2}	1
{1 3}	2
{1 5}	1
{2 3}	2
{2 5}	3
{3 5}	2

L_2

itemset	sup
{1 3}	2
{2 3}	2
{2 5}	3
{3 5}	2

C_3

itemset
{2 3 5}

L_3

itemset	sup
{2 3 5}	2

4、① Generating Candidates (字母按顺序)

ex：$L_3 = \{abc, abd, acd, ace, bcd\}$

i self-joining $L_3 \bowtie L_3$

abc, abd → abcd ; acd, ace → acde

⇒ {abcd, acde}

ii Pruning 剪枝

abcd 可分为 abc, abd, acd, bcd，它们在 L_3 中都有，abcd保留

acde 可分为 acd, ace, ade, cde，ade不在 L_3 中，删去acde

⇒ $C_4 = \{abcd\}$ → final step

② Rule Generation 若 conf ≥ min_conf → 这是强关联规则 针对 strong association

$conf(s \Rightarrow L-s) = \frac{support(L)}{support(s)}$ (L是某个频繁项集，s是L中的一个非空真子集，s与L-s互补成L)

ex：L = {2,3,5} ⇒ L中的all非空真子集：{2} {3} {5} {2,3} {2,5} {3,5}

逐条计算置信度：{2}⇒{3,5} {3}⇒{2,5} {5}⇒{2,3}

{3,5}⇒{2} {2,5}⇒{3} {2,3}⇒{5}

删去不达标的 (< min conf)，保留达标的 (≥ min_conf)

三、Comments on Apriori

(一) Bottlenecks

1、Candidate Generation：Huge candidate sets

2、Multiple scans of DB

(二) Methods to improve Apriori's Efficiency

1、Hash-based itemset counting

2、Transaction Reduction

3、Partitioning：先在分区内挖局部频繁项；再在全库验证真实支持度

4、Sampling 抽样法

5、Dynamic itemset counting：只有子集预估频繁时，才新增候选

四、Criticism to support and confidence

correlation ≠ association

1、① confidence = $P(Y|X)$，没有考虑后件Y的全局基础概率 $P(Y)$

② 高置信度 ≠ X会促进Y；有可能条件概率反而会低于全局基线，造成误导

ex：A = 运动 B = 补给饮料

	A	$\bar{A}$
B	200	175
$\bar{B}$	100	25

⇒ $P(B) = 75\%$
$P(B|A) = 66.7\%$ 因为运动而补补饮料的反而少？

2、new measures —— Interest (Lift)

$Interest(A,B) = Lift(A \Rightarrow B) = \frac{P(A \cap B)}{P(A) \cdot P(B)}$

Interest > 1：正相关，A出现，B更易发生

Interest = 1：相互独立，AB间无关

Interest < 1：负相关，A出现，B更难发生

一、Sequential Pattern Mining

ARM1：non-sequential
ARM2：sequential

1、给定DB，找出满足用户指定的最小支持度的序列模式

2、Frequent sequence

由一组项集 itemset，而不是单个 item，按时间先后顺序排列组成．$<s_1, s_2, \cdots, s_n>$

ex：<{phone, phone case}, {watch}, {Balance Wheel & Drone}> → 以事务ID为单位

普通关联规则：transaction 内挖掘，不关心时间顺序；看同一笔交易里，同时出现的商品

↳ 单粒度：从 customer ID 与时间为单位 ex：customer i + time i = item i + item j

序列模式挖掘：跨事务，按时间顺序挖掘，重点先后顺序

↳ 单粒度：从 customer ID 为单位 ex：customer i = <(item i + item j) + (item k)>

3、支持度 Support

序列 support = $\frac{\text{支持该序列的客户数}}{\text{总客户数 (not 总 transaction 数)}}$

4、Sequential Association Rule Mining Steps

① Sort Phase：main-key：Customer ID；secondary-key：time

② Frequent Itemset Phase：用ARM1的方法 Apriori $C_k \to L_k \to C_{k+1}$

有多少 transaction，包含这个 itemsets，没有时间先后

⇒ output：哪些结果满足 min_support，并且 mapping

③ Transformation Phase → 这里不是为了整合 itemsets (C_{k+n})

结合 Phase 2 的 output，重新写 Phase 1 表格，并且用 mapping 形式书写

* <{40, 70}> ⇒ <{40, 70, (40, 70)}>

④ Sequence Phase 用ARM2，新的方式 AprioriAll $C_k \to L_k \to C_{k+1}$

有多少个 customer，他的整条时序里包含这个子序列，跨 transaction，有严格时间先后

(1) how to generate C_{k+1}?

ex：$L_3 = \{abc, abd, acd, ace, bcd\}$ 还有自己和自己拼接

i self-joining $L_3 \times L_3$

abc & abd → abcd or abdc or abcc, abdd；
acd & ace → acde or aced or acdd, acee；

区别 ARM1 → 必须按照字母顺序
ARM2，就是要多重组合

ii Pruning

abde 中 bde
abce 中 bce 都不在 L_3 中；
abec 中 bec

而 abc, bcd 都在 L_3 中 ⇒ $C_4 = \{abcd\}$

⑤ Maximum Phase

(1) Definition 1：序列包含，a sequence is contained in another sequence

i 下标严格递增 → 只有 frequent sequence 才有这样的性质

ii 同一位置里，前面 sequence 中 itemset 是后面 sequence 的子集

$S = <a_1, a_2, \cdots, a_n>$ $T = <b_1, b_2, \cdots, b_m>$

ex：$S_1 = <3, (4,5), 8>$，$S_2 = <8, (4,5), 3>$，$T = <7, (3,8), 9, (4,5,6), 8>$

$S_1 \in T$，但 $S_2 \notin T$

(2) Definition 2：极大频繁序列，maximum sequence

ex：<1>, <1,2>, <1,2,3,4>；<1,2,3,4>是极大

Phase 2：对象：transaction

support：有多少 transaction，包含这个 itemsets，没有时间先后

ex：{1,5}；看有多少 itemsets 同时有1和5

都是前 k-2 个相同，对第 k-1 个重排
apriori：顺序×1 aprioriall：乱序×2

Phase 4：对象：整条 customer 时序序列

support：有多少个 customer，他的整条时序里包含这个子序列，跨 transaction，有严格时间先后

ex：<1,2>；看有多少 customer 满足：早事物出现1，晚事物出现2

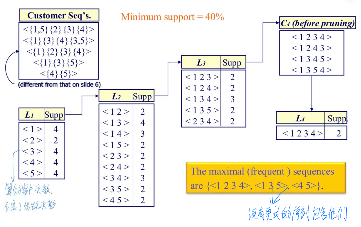
5、Forming the Sequential Association Rule

<a,b,c,d> ~~ac ⇒ bd~~ —一定不能交换 顺序

$S_f \Rightarrow S_t$

$conf(S_f \Rightarrow S_t) = \frac{Support(S)}{Support(S_f)} \geq min\_conf$；满足 strong sequence association rule

An example for step 4: Sequence Phase



Minimum support = 40%

Customer Seq's.

Customer Seq's.
<{1,5} {2} {3} {4}>
<{1} {3} {4} {3,5}>
<{1} {2} {3} {4}>
<{1} {3} {5}>
<{4} {5}>

(different from that on slide 6)

L_1	Supp
< 1 >	4
< 2 >	2
< 3 >	4
< 4 >	4
< 5 >	4

算的客户次数
不是了出现次数

L_2	Supp
< 1 2 >	2
< 1 3 >	4
< 1 4 >	3
< 1 5 >	2
< 2 3 >	2
< 2 4 >	2
< 3 4 >	3
< 3 5 >	2
< 4 5 >	2

L_3	Supp
< 1 2 3 >	2
< 1 2 4 >	2
< 1 3 4 >	3
< 1 3 5 >	2
< 2 3 4 >	2

C_4 (before pruning)
< 1 2 3 4 >
< 1 2 4 3 >
< 1 3 4 5 >
< 1 3 5 4 >

L_4	Supp
< 1 2 3 4 >	2

The maximal (frequent) sequences are {<1 2 3 4>, <1 3 5>, <4 5>}.

没有更长的序列包含他们

二、More Association Analysis

(一) Mining Multiple-Level/Generalized Association Rules —— Algorithm Basic

To solve Redundancy Problem —— extended transaction

transaction non-sequential，看 itemset 里情况

Original Transactions T

Transaction	Items bought
100	Shirt
200	Jacket, Hiking Boots
300	Ski Pants, Hiking Boots
400	Shoes
500	Shoes
600	Jacket

① form extended transactions → 把全部祖先类别加进去

Extended Transactions T'

Transaction	Items bought
100	Shirt, (Clothes)
200	Jacket, (Outerwear, Clothes,), Hiking Boots, (Footwear)
300	Ski Pants, (Outerwear, Clothes,) Hiking Boots, (Footwear)
400	Shoes, (Footwear)
500	Shoes, (Footwear)
600	Jacket, (Outerwear, Clothes)

(minsup=30%, minconf=60%)

② find frequent itemsets 在T'上跑 Apriori 目标：transaction

Frequent Itemsets

Itemset	Support
{Jacket}	2
{Outerwear}	3
{Clothes}	4
{Shoes}	2
{Hiking Boots}	2
{Footwear}	4
{Outwear, Hiking Boots}	2
{Clothes, Hiking Boots}	2
{Outerwear, Footwear}	2
{Clothes, Footwear}	2

过滤后的情况
父子项直接筛掉

③ find association rule 没有顺序要求 $L_2 = \{A, B\}$ A⇒B & B⇒A

Rules

Rule	Support	Conf.
{Outerwear}⇒{Hiking Boots}	33%	66%
{Outerwear}⇒{Footwear}	33%	66%
{Hiking Boots}⇒{Outerwear}	33%	100%
{Hiking Boots}⇒{Clothes}	33%	100%

(二) how to set minimum support?

	Uniform Support 统一支持	Reduced Support 递减支持
minsup 设置	所有层级使用同一个最小支持度	高层大品类：高 minsup；底层细分商品：更小 minsup
本例参数	L1=5%, L2=5%	L1=5%, L2=3%
Skim Milk 结果	supp=4% <5% → 过滤丢弃	supp=4% >3% → 保留
优点	实现简单，只维护1个阈值	可以捕获底层细粒度商品的关联
缺点	两难：阈值高丢底层规则；阈值低生成大量高层冗余规则	需要设置多层阈值，算法复杂

(三)

① Mult-Dimensional Association

i single：ex：buys(X, "milk") ⇒ buys(X, "bread")

ii multi-dimension 1) inter-dimension：no repeated predicates age ∧ occupation ⇒ buys
2) hybrid-dimension：repeated predicates age ∧ buys ⇒ buys

iii categorical attributes：取值是有限集合，取值之间没大小顺序

iv quantitative attributes：数值类型，自带大小顺序

② Mining Quantitative Associations

数值是 continuous 的，不可直接放入 Apriori 的 itemset，3种方式处理数值属性

i static discretization 静态离散化 ex：人为提前划分：age → [19,25], [26-35]

ii dynamic discretization 动态离散化 根据 data，自动划分

iii clustering 聚类 先做一维聚类，把相似数值聚成簇，再在簇上挖掘关联规则

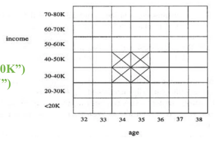
③ Quantity Association Rule

Such that, the confidence or compactness of the rules mined is maximized.

Rules to form general rules using a 2-D grid

- Example

age(X, "34-35") ∧ income(X, "30-50K")
⇒ buys(X, "high resolution TV")



④ Spatial Association Analysis

1. $is\_a(x, large\_town) \wedge intersect(x, highway) \Rightarrow adjacent\_to(x, water)$ [7%, 85%]

   含义：x是大城镇，并且与高速相交 ⇒ x毗邻水域，支持度7%，置信度85%

2. $is\_a(x, large\_town) \wedge adjacent\_to(x, georgia\_strait) \Rightarrow close\_to(x, u.s.a.)$ [1%, 78%]

   含义：x是大城镇，毗邻佐治亚海峡 ⇒ 靠近美国，支持度1%，置信度78%

一、Basic Knowledge

1、{ classification：predict class labels
  regression： predict values

2、{ model construction：train & verify
  model usage：test ⟶ 属于seen data，但在model construction中属于unseen data

3、{ supervised (classification)：label for training set
  model基于训练集学习，然后对new data做分类预测
  unsupervised (clustering)：no label for training set
  给定观测/测量data，目标是挖掘data内部天然存在类别 or cluster结构

4、issue for Data Preparation：① data cleaning
② feature engineering (relevance analysis 剔除冗余无关特征)
③ data transformation 对数据(文字→数字)泛化，归一化处理

5、Evaluating classification/regression methods
① Predictive accuracy ② Speed & Scalability ③ Robustness ④ Interpretability
⑤ Scalability 可伸缩性：efficiency in disk-resident database
⑥ Goodness of rules：ex：decision tree size；compactness of classification rules

二、DT (? how classification & regression)

1、Basic knowledge

① Structure
i flow-chart-like tree structure iii branch：属性测试的一个输出结果
ii internal node：对某个属性做测试 iv leaf node：某class labels or class distribution

② DT Generation
i tree construction：初始时，all训练样本放在根节点；根据挑选出的属性，递归对样本做划分
ii tree pruning：识别并移除由噪点、异常点产生的分支，来减轻过拟合

ex：如果目标output是"是否买computer"，那DT一定先是对各属性的筛选，
然后root一定是对"是否买computer"的回答：yes or no

2、Algorithm for DT —— Greedy Algorithm (当前最优选择 ⟶ not全局最优)
① top-down recursive divide & conquer manner
② use information gain (heuristic or statistical manner)

3、How to split when in the tree.
① Information Gain —— from Entropy (classification)
② Gini Index (CART：classification + regression)

4、Solve Overfitting
① pre-pruning： threshold value (阈值) { too small：stop early，lack of fit
  too large：overfitting

② post-pruning：先生成全树 ⟶ 自底向上剪枝，替换成叶子节点 ⟶ 验证集评估，选最优版本
use validation data to decide which is the "best pruned tree"

5、dataset无法被DT完美分类时，how can we do?
① for dataset：data cleaning，feature engineering，more dataset
② for model：i pre/post-pruning
ii ensembling
iii adjust the ginseng 调参
iv 换属性指标：information gain ⟶ gini index

三、Information Gain —— from Entropy

$$I(a,b,c) = -\frac{a}{a+b+c}\log_2\frac{a}{a+b+c} - \frac{b}{a+b+c}\log_2\frac{b}{a+b+c} - \frac{c}{a+b+c}\log_2\frac{c}{a+b+c}$$

$$E(A) = \sum_{i=1}^{v}\frac{p_i+n_i}{p+n} I(p_i, n_i)$$

$Gain(A) = I(a,b,c) - E(A)$ 每次挑选Gain大的分类，也是Entropy小的分类

四、Regression Tree 每次挑选SSE小的分类：划分后，现象：子node一定小于父node

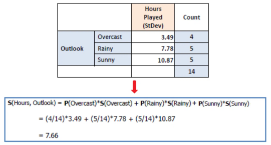
1、① output node是数值 ② predict的结果是所在目标的均值
③ 用 $SSE = \sum_{i \in node}(y_i - \bar{y})^2$ 或 standard deviation $sd = \sqrt{\frac{\sum(y_i-\bar{y})^2}{n}}$ 来split node
④ 用 $RSME = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}$，性能评估
⑤ 用 $CV = \frac{sd}{\bar{x}}$ to determine branching termination

2、公式 挑选S(T,X)最小的，SDR(T,X)最大的

$SDR(T,X) = S(T) - S(T,X)$

$S(T)$：对于全部样本求sd

$S(T,X) = \sum_{c \in X} P(c) \cdot S(c)$



		Hours Played (StDev)	Count
Outlook	Overcast	3.49	4
	Rainy	7.78	5
	Sunny	10.87	5
			14

S(Hours, Outlook) = P(Overcast)*S(Overcast) + P(Rainy)*S(Rainy) + P(Sunny)*S(Sunny)

= (4/14)*3.49 + (5/14)*7.78 + (5/14)*10.87

= 7.66

* i depth for tree (0 ~ n-1个属性)

ii classification rule：if A(& B & C) then D(class attribute)

iii regression tree —— 正常按照DT画，只不过label处理方式不同
↓
用SDR split node
RT正常经过all attributes (除非pre-pruning)
最后"label"要么是具体value，要么取平均

iv class attribute：目标属性